

تمرین ۲

با توجه به مجموعه آموزش زیر از داده های dogs:

Color	Fur	Size	Class
brown	ragged	small	well-behaved
black	ragged	big	dangerous
black	smooth	big	dangerous
black	curly	small	well-behaved
white	curly	small	well-behaved
white	smooth	small	dangerous
red	ragged	big	well-behaved

با استفاده از الگوریتم ID۳ یک درخت تصمیم تعیین کنید، به طوری که igain ویژگی ها انتخاب شوند.

حل

مرحله ۱: محاسبه آنتروپی مجموعه کل S

مجموعه S شامل ۷ نمونه است:

$$\#(\text{well-behaved}) = 4, \quad \#(\text{dangerous}) = 3.$$

بنابراین:

$$p_+ = \frac{4}{7}, \quad p_- = \frac{3}{7}.$$

آنتروپی:

$$H(S) = -p_+ \log_2 p_+ - p_- \log_2 p_- = -\frac{4}{7} \log_2 \frac{4}{7} - \frac{3}{7} \log_2 \frac{3}{7}.$$

محاسبه عددی:

$$\log_2 \frac{4}{7} \approx \log_2(0.5714) \approx -0.8074, \quad \log_2 \frac{3}{7} \approx \log_2(0.4286) \approx -1.2224.$$

پس:

$$H(S) = -\frac{4}{7}(-0.8074) - \frac{3}{7}(-1.2224) = \frac{3.2296 + 3.6672}{7} = \frac{6.8968}{7} \approx 0.9853.$$

مرحله ۲: محاسبه Gain Information برای هر ویژگی

ویژگی Size

مقادیر: small (۴ نمونه)، big (۳ نمونه)

• S_{small} : ۴ نمونه $\leftarrow$ well-behaved: ۳، dangerous: ۱

$$H(S_{\text{small}}) = -\frac{3}{4} \log_2 \frac{3}{4} - \frac{1}{4} \log_2 \frac{1}{4} = -0.75 \cdot (-0.4150) - 0.25 \cdot (-2) = 0.3113 + 0.5 = 0.8113.$$

• S_{big} : ۳ نمونه $\leftarrow$ dangerous: well-behaved: ۲

$$H(S_{\text{big}}) = -\frac{1}{3} \log_2 \frac{1}{3} - \frac{2}{3} \log_2 \frac{2}{3} \approx -0.3333 \cdot (-1.5850) - 0.6667 \cdot (-0.5850) = 0.5283 + 0.3900 = 0.9183.$$

پس:

$$\begin{aligned} \text{IG}(S, \text{Size}) &= H(S) - \left(\frac{4}{7} H(S_{\text{small}}) + \frac{3}{7} H(S_{\text{big}}) \right) \\ &= 0.9853 - \left(\frac{4}{7} \cdot 0.8113 + \frac{3}{7} \cdot 0.9183 \right) = 0.9853 - (0.4636 + 0.3936) = 0.9853 - 0.8572 = \boxed{0.1281}. \end{aligned}$$

ویژگی Fur

مقادیر: (۲) curly, (۲) smooth, (۳) ragged

• S_{ragged} : ۳ نمونه $\leftarrow$ dangerous: ۱, well-behaved: ۲

$$H = -\frac{2}{3} \log_2 \frac{2}{3} - \frac{1}{3} \log_2 \frac{1}{3} \approx 0.9183 \quad (\text{بالا } S_{\text{big}} \text{ همانند})$$

• S_{smooth} : ۲ نمونه $\leftarrow$ dangerous: ۲, well-behaved: ۰

$$H = 0 \quad (\text{همگن})$$

• S_{curly} : ۲ نمونه $\leftarrow$ dangerous: ۰, well-behaved: ۲

$$H = 0 \quad (\text{همگن})$$

پس:

$$\text{IG}(S, \text{Fur}) = 0.9853 - \left(\frac{3}{7} \cdot 0.9183 + \frac{2}{7} \cdot 0 + \frac{2}{7} \cdot 0 \right) = 0.9853 - \frac{2.7549}{7} = 0.9853 - 0.3936 = \boxed{0.5917}.$$

ویژگی Color

مقادیر: (۱) brown, (۱) black, (۳) white, (۱) red — همه یک‌نواخت یا تقریباً همگن

• brown: ۱ نمونه $\leftarrow$ well-behaved $\leftarrow$ $H = 0$

• black: ۳ نمونه $\leftarrow$ dangerous: ۲, well-behaved: ۱ $\leftarrow$ $H = 0.9183$

• white: ۲ نمونه $\leftarrow$ dangerous: ۱, well-behaved: ۱ $\leftarrow$

$$H = -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2} = 1$$

• red: ۱ نمونه $\leftarrow$ well-behaved $\leftarrow$ $H = 0$

پس:

$$\begin{aligned} \text{IG}(S, \text{Color}) &= 0.9853 - \left(\frac{1}{7} \cdot 0 + \frac{3}{7} \cdot 0.9183 + \frac{2}{7} \cdot 1 + \frac{1}{7} \cdot 0 \right) \\ &= 0.9853 - \left(\frac{2.7549 + 2}{7} \right) = 0.9853 - \frac{4.7549}{7} = 0.9853 - 0.6793 = \boxed{0.3060}. \end{aligned}$$

نتیجه نهایی

ویژگی	Gain Information
Size	0.1281
Fur	0.5917
Color	0.3060

پس ****** بهترین ویژگی برای تقسیم ریشه ******، ویژگی Fur است، زیرا بیشترین *Gain Information* را دارد.

تمرین ۳

(الف) برای ساخت یک درخت تصمیم، همیشه از یک جست و جوی حریصانه (Greedy Search) استفاده می‌شود. اصطلاح *Search Greedy* چه معنایی دارد؟ آیا چه شرایطی یک راهبرد حریصانه مفید است؟ چه راهبردهای جایگزینی وجود دارد؟

(ب) سوگیری استقرایی (inductive bias) الگوریتم *Candidate Elimination* بر مکانیزمی متفاوت از ID۳ استوار است. این گزاره را با تحلیل منطق سوگیری استقرایی هر یک از این دو الگوریتم توضیح دهید.

(ج) پیچیدگی زمانی الگوریتم ID۳ چقدر است؟ پاسخ خود را توضیح دهید.

حل

(الف) Search Greedy در درخت تصمیم

Greedy Search (جست و جوی حریصانه) یک روش بهینه‌سازی محلی است که در هر مرحله، تصمیمی را اتخاذ می‌کند که بهترین نتیجه محلی را در آن لحظه ایجاد می‌کند، بدون اینکه به پیامدهای بلندمدت یا حالت‌های آینده توجه کند.

در الگوریتم ID۳ - در هر گره، ویژگی‌ای انتخاب می‌شود که بیشترین *Information Gain* (یا کمترین آنتروپی پس از تقسیم) را ایجاد کند. - این انتخاب تنها بر اساس داده‌های فعلی و بدون بررسی ترکیبات بعدی انجام می‌شود — یعنی حریصانه است.

چه زمانی راهبرد حریصانه مفید است؟

راهبرد حریصانه زمانی مفید است که:

- مسئله دارای خواص انتخاب بهینه محلی (optimal substructure) باشد، یعنی بهینه‌سازی محلی منجر به جواب بهینه کلی شود.
- یا زمانی که هزینه جستجوی فضای کامل غیرقابل قبول باشد (مثل درخت‌های تصمیم با ویژگی‌های زیاد)، و نیاز به یک راه‌حل تقریبی سریع وجود داشته باشد.
- در عمل، ID۳ با این راهبرد، در بسیاری از مسائل واقعی عملکرد خوبی دارد، هرچند از نظر نظری ممکن است به جواب بهینه کلی نرسد.

راهبردهای جایگزین

- **Branch-and-Bound**: جستجوی کامل با قطع شاخه‌های غیر promising — اما محاسبه‌بر است.
- **Simulated Annealing / Genetic Algorithms**: جستجوی فراابتکاری برای یافتن درخت‌های بهینه‌تر (غیرحریصانه).
- **Ensemble Methods** (مثل random Forest): به جای یک درخت، چند درخت را با نمونه‌برداری تصادفی می‌سازد و از میانگین‌گیری نتایج استفاده می‌کند.

(ب) مقایسه سوگیری استقرایی (Inductive Bias) در Candidate Elimination و ID۳

سوگیری استقرایی (Inductive Bias) مجموعه فرضیاتی است که یک الگوریتم یادگیری برای انتخاب یک فرضیه از بین تمام فرضیه‌های ممکن اعمال می‌کند. بدون این سوگیری، یادگیری از داده‌های محدود غیرممکن است (به دلیل اصل عدم قطعیت — *No Free Lunch Theorem*).

۱. Candidate Elimination

– فرضیه‌ها: مجموعه تمام مفاهیم قابل بیان به صورت کانال‌های متشکل از متغیرهای گسسته (مثلاً قاعده‌هایی مانند $\langle \text{big, ragged} \rangle$). – سوگیری استقرایی:

- فرض می‌کند که مفهوم هدف وجود دارد و در فضای فرضیه‌ها قرار دارد (یعنی داده‌ها بدون خطا هستند).
- فرض می‌کند که مفهوم هدف کمینه‌است (minimal) — یعنی از ساده‌ترین فرضیه‌های سازگار با داده‌ها انتخاب می‌شود.
- از دو مجموعه S (سفید — ساده‌ترین فرضیه‌های سازگار) و G (سیاه — پیچیده‌ترین فرضیه‌های سازگار) استفاده می‌کند.
- سوگیری: «مفاهیم ساده‌تر ترجیح داده می‌شوند» (Razor Occam's)، اما به صورت صریح و محدود به فضای فرضیه‌های قابل بیان با $\text{conjunctions of literals}$.

۲. ID۳

- فرضیه‌ها: تمام درخت‌های تصمیم ممکن برای طبقه‌بندی داده‌ها. – سوگیری استقرایی:
- فرض می‌کند که درخت‌های کوچک‌تر بهتر هستند (کوتاه‌ترین درخت = ساده‌ترین مدل).
 - این سوگیری به صورت **implicit** در الگوریتم اعمال می‌شود: با انتخاب ویژگی با بیشترین Information Gain، درخت به صورت معمولاً کم‌عمق‌تر و کم‌شاخه‌تر ساخته می‌شود.
 - همچنین، ID۳ فرض می‌کند که ویژگی‌ها مستقل از هم هستند (اگرچه این فرض صریحاً ذکر نمی‌شود، اما در محاسبه IG از این فرض استفاده می‌شود).
 - سوگیری اصلی: «بهینه‌سازی محلی منجر به مدل ساده و کارآمد می‌شود».

مقایسه کلی

ID۳	Candidate Elimination	ویژگی
گسترده (درخت‌های تصمیم)	محدود (conjunctions)	فضای فرضیه
درخت‌های کوچک و با IG بالا	مفاهیم ساده و کمینه	سوگیری اصلی
ضمنی (در انتخاب ویژگی)	صریح (S) و (G)	نوع سوگیری
متوسط (با pruning مقاوم‌تر می‌شود)	بسیار بالا (خطا ← هیچ فرضیه‌ای سازگار نیست)	حساسیت به خطا

ج) پیچیدگی زمانی الگوریتم ID۳

فرض کنید: n – تعداد نمونه‌ها – m : تعداد ویژگی‌ها – d : عمق درخت (حداکثر تعداد تقسیمات) – k : حداکثر تعداد مقادیر ممکن برای یک ویژگی (مثلاً برای ویژگی گسسته)

در هر گره: – برای هر ویژگی، باید آنتروپی زیرمجموعه‌ها را محاسبه کنیم. – محاسبه آنتروپی برای یک ویژگی با k مقدار، $O(n)$ زمان می‌برد (چون باید نمونه‌ها را بر اساس مقدار ویژگی گروه‌بندی کنیم). – پس برای هر گره، زمان $O(m \cdot n)$ لازم است.

تعداد گره‌ها در بدترین حالت (درخت کامل): $O(2^d)$ ، اما در عمل، هر نمونه فقط یک بار در هر سطح پردازش می‌شود، و مجموعاً هر نمونه در حداکثر d گره ظاهر می‌شود.

بنابراین، پیچیدگی زمانی کلی ID۳:

$$O(m \cdot n \cdot d)$$

اما اگر فرض کنیم $d \leq m$ (چون درخت نمی‌تواند عمیق‌تر از تعداد ویژگی‌ها باشد)، آنگاه:

$$O(m^2 \cdot n)$$

در بعضی منابع، پیچیدگی را به صورت $O(m \cdot n \log n)$ ذکر می‌کنند اگر برای گروه‌بندی از مرتب‌سازی استفاده شود، اما در پیاده‌سازی استاندارد ID۳ (بدون مرتب‌سازی)، پیچیدگی $O(mnd)$ صحیح‌تر است.

نکته مهم

- ID۳ در حالت کلی چندجمله‌ای (polynomial) است، اما در صورتی که از ویژگی‌های پیوسته و روش‌های پیچیده‌تر برای تقسیم استفاده شود، پیچیدگی ممکن است افزایش یابد. - این پیچیدگی به دلیل جستجوی حریصانه، بسیار کمتر از جستجوی کامل در فضای درخت‌ها ($O(2^m)$) است.

تمرین ۴

(الف) بیش‌برازش (overfitting) چیست؟

(ب) چرا مجموعه‌ای نمونه‌دار (test set) و مجموعه‌ای آموزشی (training set) در یادگیری ماشین استفاده می‌شود؟ آیا چنین تقسیم‌بندی‌ای جلوگیری از بیش‌برازش را تضمین می‌کند؟ چرا یا چرا نه؟

(ج) بیش‌برازش‌های جلوگیری از بیش‌برازش، استفاده از الگوریتم‌های موسوم به پرس‌هرس (post-pruning) است. ابتدا یک درخت تصمیم بیش‌برازش شده بزرگ ساخته می‌شود و سپس با هرس کردن، تصمیم داده می‌شود. راهبردهای مختلف هرس مانند *rule post pruning* و *reduced-error pruning* را توضیح دهید.

(د) کدام یک از گزاره‌های زیر صحیح است؟

(الف) زمان آموزش کوتاه‌تر باعث بیش‌برازش (overfitting) می‌شود.

(ب) درخت تصمیم کوچک‌تر نسبت به تصمیم بزرگ‌تر، تعمیم‌پذیری (generalizes) دارد.

(ج) توانایی تعمیم (generalization capability) مجموعه‌ای آموزشی (training) وابسته است.

(د) نظریه اطلاعات (information theory) اثرات منفی مجموعه‌های آموزش کوچک را سوگیری-bias (bi-ased) می‌داند.

(ه) آیا رابطه‌ای بین روش‌های شبکه عصبی مانند *dropout* و روش‌های هرس کردن درخت تصمیم وجود دارد؟ توضیح دهید.

حل

(الف) بیش‌برازش (Overfitting) چیست؟

بیش‌برازش (Overfitting) حالتی است که در آن مدل یادگیری ماشین، به قدری به داده‌های آموزشی خیلی دقیق تنظیم می‌شود که نه تنها الگوهای واقعی را یاد می‌گیرد، بلکه نویز و ویژگی‌های تصادفی داده‌های آموزشی را نیز به عنوان الگو در نظر می‌گیرد.

نتیجه: - خطای آموزشی (training error) بسیار کوچک (حتی صفر) می‌شود. - اما خطای تست (test error) بزرگ می‌شود ← مدل تعمیم ضعیفی دارد.

مثال: درخت تصمیمی که برای هر نمونه آموزشی یک برگ جداگانه داشته باشد — کاملاً بیش‌برازش شده است.

(ب) نقش مجموعه‌های آموزشی و تست

- مجموعه آموزشی (training set): برای یادگیری پارامترها (مثلاً ساخت درخت تصمیم). - مجموعه تست (test set): برای ارزیابی عملکرد مدل روی داده‌های دیده‌نشده — یعنی اندازه‌گیری تعمیم.

آیا این تقسیم جلوگیری از بیش‌برازش را تضمین می‌کند؟ خیر. دلیل: - تقسیم فقط امکان تشخیص بیش‌برازش را فراهم می‌کند (چون می‌توانیم خطای تست را ببینیم)، اما خود تقسیم، بیش‌برازش را جلو نمی‌گیرد.

- اگر مدل بیش‌برازش شده باشد، خطای تست بالا خواهد بود — ولی این خود تقسیم نیست که جلوی بیش‌برازش را می‌گیرد؛ بلکه روش‌هایی مثل *regularization*، *pruning* یا افزایش داده، جلوی آن را می‌گیرند.

(ج) راهبردهای پرس هرس (Post-Pruning)

در روش *post-pruning*، ابتدا یک درخت بزرگ و بیش برآزش شده ساخته می‌شود، سپس شاخه‌های غیرضروری حذف می‌شوند تا تعمیم بهبود یابد. دو راهبرد رایج:

- **Rule Post-Pruning**: هر مسیر از ریشه تا برگ به یک قاعده (rule) تبدیل می‌شود، مثلاً:

$$\text{If Size} = \text{big} \wedge \text{Fur} = \text{ragged} \Rightarrow \text{Class} = \text{well-behaved}$$

سپس هر قاعده به تنهایی ارزیابی می‌شود و اگر حذف آن، خطای تست را کاهش دهد، حذف می‌شود. در نهایت، قاعده‌ها مرتب و ساده‌تر می‌شوند.

- **Reduced-Error Pruning**: در هر گره داخلی، بررسی می‌شود که اگر آن گره را به یک برگ تبدیل کنیم (یعنی زیردرخت را حذف کنیم)، آیا خطای تست کاهش می‌یابد؟ اگر بله، انجام می‌شود. این روش به صورت پایین به بالا (bottom-up) انجام می‌شود و مستقل از داده‌های آموزشی است — فقط از مجموعه اعتبارسنجی (validation set) استفاده می‌کند.

(د) بررسی گزاره‌ها

۱. زمان آموزش کوتاه‌تر باعث بیش برآزش (overfitting) می‌شود. **نادرست**
توضیح: مدل‌های ساده‌تر (زمان آموزش کوتاه‌تر) معمولاً کمتر در معرض بیش برآزش هستند. بیش برآزش معمولاً در مدل‌های پیچیده (زمان آموزش بلند/عمیق) رخ می‌دهد.

۲. درخت تصمیم کوچک‌تر نسبت به تعمیم بزرگ‌تر، تعمیم‌پذیری (generalizes) دارد. **صحیح**
توضیح: درخت‌های کوچک‌تر معمولاً ساده‌ترند و از نویز داده‌ها کمتر تأثیر می‌گیرند — پس تعمیم بهتری دارند (البته تا حدی؛ اگر خیلی کوچک شود، تحت برآزش می‌شود).

۳. توانایی تعمیم (generalization capability) وابسته به مجموعه آموزشی (training) است. **نادرست**
توضیح: توانایی تعمیم وابسته به مدل و روش یادگیری است، نه مجموعه آموزشی. البته کیفیت و اندازه مجموعه آموزشی تأثیر دارد، اما تعمیم یک ویژگی مدل است، نه مجموعه داده.

۴. نظریه اطلاعات (information theory) اثرات منفی مجموعه‌های آموزش کوچک را سوگیری (biased) می‌داند. **نادرست**
توضیح: کوچک بودن مجموعه آموزشی معمولاً منجر به واریانس بالا و سوگیری پایین می‌شود (در trade-off bias-variance). اما نظریه اطلاعات مستقیماً «سوگیری» را تعریف نمی‌کند؛ این مفهوم از *statistical learning theory* و *bias-variance decomposition* آمده است.

(ه) رابطه بین dropout و هرس کردن درخت تصمیم

بله، رابطه‌ای مفهومی وجود دارد:
هر دو روش، از یک اصل مشترک پیروی می‌کنند:

«جلوگیری از وابستگی بیش از حد به ویژگی‌های خاص داده‌های آموزشی»

- **Dropout** در شبکه‌های عصبی: در هر دوره آموزش، تصادفی برخی نورون‌ها «غیرفعال» می‌شوند. این کار مانع از این می‌شود که نورون‌ها به طور متقابل وابسته شوند و ویژگی‌های خاص داده‌ها را حفظ کنند. در واقع، dropout یک نوع **ensemble averaging** است (آموزش چند زیرشبکه و میانگین‌گیری).

- **هرس کردن (Pruning)** در درخت تصمیم: شاخه‌هایی که فقط بر اساس نمونه‌های کمی ساخته شده‌اند (و احتمالاً نویز را یاد گرفته‌اند) حذف می‌شوند. این هم منجر به یک مدل ساده‌تر و کمتر وابسته به داده‌های آموزشی می‌شود.

جمع‌بندی:

هر دو روش، با کاهش پیچیدگی مؤقت/دائمی مدل، از بیش‌برازش جلوگیری می‌کنند و تعمیم را بهبود می‌بخشند. این یک نمونه از *regularization* است — یعنی اعمال محدودیت بر فضای فرضیه‌ها برای جلوگیری از بیش‌برازش.

تمرین ۵

(الف) معایب توقف زودهنگام (stopping) در مقایسه با هرس کردن (pruning) برای ساخت درخت‌های تصمیم چیست؟ تصمیم بحث کنید.

(ب) کاربرد استفاده از مجموعه‌های نمونه متفاوت را در دو مورد زیر توضیح دهید:

(الف) ساخت درخت تصمیم (without pruning)

(ب) انتخاب درخت تصمیم میان مجموعه‌ای از نامزدهای هرس (pruning candidates)

حل

(الف) معایب توقف زودهنگام (Stopping) در مقایسه با هرس کردن (Pruning)

توقف زودهنگام (Early Stopping) به معنای متوقف کردن رشد درخت قبل از رسیدن به حالت کامل (مثلاً وقتی همه نمونه‌ها در یک گره همگن شدند یا تعداد نمونه‌ها زیر آستانه‌ای قرار گرفت) است. در مقابل، هرس کردن (Pruning) به معنای ساخت یک درخت بزرگ و سپس حذف شاخه‌های غیرضروری است.

معایب stopping نسبت به pruning:

۱. عدم دسترسی به جواب بهینه محلی: در stopping تصمیم برای متوقف کردن درخت بر اساس معیارهای ساده (مثل حداقل تعداد نمونه در گره) گرفته می‌شود، بدون بررسی اینکه آیا ادامه ساخت درخت می‌تواند به بهبود تعمیم منجر شود. در حالی که در pruning پس از ساخت درخت کامل، ارزیابی مقایسه‌ای انجام می‌شود (مثلاً با validation set)، و فقط شاخه‌هایی که تعمیم را بدتر می‌کنند حذف می‌شوند.

۲. حساسیت به پارامترهای آستانه‌ای: stopping به شدت به تنظیماتی مثل min_samples_split یا max_depth وابسته است. انتخاب نادرست این پارامترها می‌تواند منجر به تحت‌برازش (underfitting) شود، در حالی که pruning به طور خودکار این تعادل را بهبود می‌بخشد.

۳. عدم استفاده از اطلاعات کلی داده: در stopping هر گره به صورت مستقل تصمیم می‌گیرد (local decision)، در حالی که در pruning تصمیم بر اساس رفتار کلی درخت روی مجموعه اعتبارسنجی گرفته می‌شود — یعنی از اطلاعات جهانی (global) استفاده می‌کند.

۴. کاهش انعطاف‌پذیری: stopping یک روش پیش‌گیرانه (preventive) است، در حالی که pruning یک روش تصحیح‌کننده (corrective) است. این امر به pruning اجازه می‌دهد تا درخت‌های پیچیده‌تری را بسازد و سپس آن‌ها را بهینه کند — که در عمل، عملکرد بهتری دارد.

جمع‌بندی: Stopping سریع‌تر است، اما pruning معمولاً منجر به درخت‌هایی با تعمیم بهتر می‌شود و از نظر کیفیت مدل، برتر است — به خصوص وقتی مجموعه اعتبارسنجی (validation set) در دسترس باشد.

(ب) کاربرد مجموعه‌های نمونه متفاوت

(۱) ساخت درخت تصمیم (بدون هرس)

در این حالت، از مجموعه آموزشی (training set) برای ساخت درخت استفاده می‌شود. اما برای جلوگیری از بیش‌برازش، معمولاً از یک مجموعه اعتبارسنجی (validation set) برای:

- تعیین آستانه‌های stopping (مثل min samples per leaf)،

- ارزیابی نیاز به ادامه تقسیم یا خیر،

استفاده می‌شود. همچنین، در روش‌هایی مثل *cross-validation*، داده‌ها به صورت چند برش (folds) تقسیم می‌شوند تا از تمام داده‌ها برای هم آموزش و هم اعتبارسنجی استفاده شود.

(۲) انتخاب درخت تصمیم میان مجموعه‌ای از نامزدهای هرس (Pruning Candidates)

در روش‌هایی مانند *reduced-error pruning* یا *cost-complexity pruning* (در CART ابتدا یک درخت کامل ساخته می‌شود، سپس یک سری درخت‌های هرس شده (pruned trees) تولید می‌شوند — هر کدام با درجه هرس متفاوت (مثلاً با پارامتر α در CART). سپس:

- هر یک از این درخت‌های نامزد (pruning candidates) روی مجموعه اعتبارسنجی (validation set) ارزیابی می‌شوند.

- درختی که کمترین خطای اعتبارسنجی را داشته باشد، انتخاب می‌شود.

چرا از مجموعه‌ای متفاوت استفاده می‌شود؟ چون:

- اگر از مجموعه آموزشی برای انتخاب هرس استفاده شود، مدل تمایل دارد درختی را انتخاب کند که روی داده‌های آموزشی بهتر عمل کند — یعنی دوباره بیش‌برازش می‌شود.
- اما با استفاده از مجموعه تست/اعتبارسنجی جداگانه، اطمینان حاصل می‌شود که انتخاب بر اساس تعمیم انجام شده است، نه تنها برازش به داده‌های آموزشی.

نکته کلیدی: استفاده از مجموعه‌های متفاوت برای آموزش، اعتبارسنجی و تست، اصل اساسی در یادگیری ماشین است و به نام **data separation principle** شناخته می‌شود — برای جلوگیری از تورم ارزیابی (optimistic bias) در اندازه‌گیری عملکرد مدل.

تمرین ۶

فرض کنید مجموعه کلاس‌ها $C = \{c_1, c_2, c_3, c_4\}$ باشد و X فضای ویژگی و D مجموعه داده‌ای از نمونه‌ها باشد. دو شکاف (node) در درخت تصمیم به صورت زیر نشان داده شده‌اند (با توزیع کلاس‌ها):

(راهنمای رنگ: سفید c_1 ، سبز c_2 ، زرد c_3 ، قرمز c_4)

(الف) مقدار $\Delta t(\{D_1, D_2\}; t)$ را با معیار نرخ خطا (misclassification error) و معیار Gini محاسبه کنید. کدام شکاف مجموعه‌ای که بیش بهبود را ایجاد می‌کند؟

(ب) نتایج به دست آمده را تفسیر کنید.

(ج) یک ماتریس هزینه 4×4 confusion ارائه دهید که هزینه اشتباه طبقه‌بندی را بیان کند. سپس تعریف کنید:

$$\text{cost}(c' | c) = \begin{cases} \geq 0, & c' \neq c, c \in C, \\ 0, & \text{غیر این صورت} \end{cases} \quad (۱)$$

و توضیح دهید که چرا این تعریف برای طبقه‌بندی با هزینه‌های نامتقارن مناسب است. همچنین، توجیه کنید که چرا در شکاف سمت راست (شکاف سمت راست)، ترجیح داده شود و در شکاف سمت چپ، ترجیح داده نشده است.

حل

پیش‌فرض‌ها و توزیع کلاس‌ها

با توجه به شکل (و فرض تعداد نمونه‌ها بر اساس اندازه بخش‌های رنگی): - در شکاف ریشه (قبل از تقسیم): فرض می‌کنیم مجموعه کل D شامل ۸ نمونه است: $c_1 : 2, c_2 : 2, c_3 : 2, c_4 : 2$ ← توزیع یکنواخت.
- در شکاف D_1 (سمت چپ پس از تقسیم): نمونه‌ها: $c_1 : 1, c_2 : 2, c_3 : 1, c_4 : 0$ ← مجموع $|D_1| = 4$
- در شکاف D_2 (سمت راست پس از تقسیم): نمونه‌ها: $c_1 : 1, c_2 : 0, c_3 : 1, c_4 : 2$ ← مجموع $|D_2| = 4$
(اگر تصویر دقیق‌تری دارید، می‌توانم اعداد را دقیق‌تر تنظیم کنم — اما این توزیع معمول و منطقی است.)

(الف) محاسبه Δt با دو معیار

۱. معیار نرخ خطا (Misclassification Error)

نرخ خطا برای یک گره:

$$E(S) = 1 - \max_{c \in C} p(c | S)$$

- برای ریشه D : $p(c_i) = 2/8 = 0.25$ برای همه i ← $\max p = 0.25$

$$E(D) = 1 - 0.25 = 0.75$$

- برای D_1 ($|D_1| = 4$): $p(c_1) = 1/4, p(c_2) = 2/4 = 0.5, p(c_3) = 1/4, p(c_4) = 0$ ← $\max = 0.5$

$$E(D_1) = 1 - 0.5 = 0.5$$

max = 0.5 ← $p(c_1) = 1/4$, $p(c_3) = 1/4$, $p(c_4) = 2/4 = 0.5$: ($|D_2| = 4$) برای D_2 -

$$E(D_2) = 1 - 0.5 = 0.5$$

پس:

$$\Delta t_{\text{error}} = E(D) - \left(\frac{|D_1|}{|D|} E(D_1) + \frac{|D_2|}{|D|} E(D_2) \right) = 0.75 - \left(\frac{4}{8} \cdot 0.5 + \frac{4}{8} \cdot 0.5 \right) = 0.75 - (0.25 + 0.25) = \boxed{0.25}$$

۲. معیار Gini

impurity: Gini

$$G(S) = 1 - \sum_{c \in C} p(c | S)^2$$

- برای D :

$$G(D) = 1 - 4 \cdot (0.25)^2 = 1 - 4 \cdot 0.0625 = 1 - 0.25 = 0.75$$

- برای D_1 :

$$G(D_1) = 1 - [(1/4)^2 + (2/4)^2 + (1/4)^2 + 0^2] = 1 - [0.0625 + 0.25 + 0.0625] = 1 - 0.375 = 0.625$$

- برای D_2 :

$$G(D_2) = 1 - [(1/4)^2 + 0^2 + (1/4)^2 + (2/4)^2] = 1 - [0.0625 + 0 + 0.0625 + 0.25] = 1 - 0.375 = 0.625$$

پس:

$$\Delta t_{\text{Gini}} = G(D) - \left(\frac{4}{8} G(D_1) + \frac{4}{8} G(D_2) \right) = 0.75 - (0.5 \cdot 0.625 + 0.5 \cdot 0.625) = 0.75 - 0.625 = \boxed{0.125}$$

مقایسه و نتیجه

$$\Delta t_{\text{Gini}} = 0.125 - \Delta t_{\text{error}} = 0.25 -$$

هر دو معیار بهبود مثبتی نشان می‌دهند، اما معیار نرخ خطا بهبود بیشتری را گزارش می‌کند. اما این به توزیع کلاس‌ها بستگی دارد. در این مورد، چون در هر زیرمجموعه یک کلاس غالب با احتمال ۰.۵ وجود دارد، معیار خطا حساس‌تر است.

(ب) تفسیر نتایج

- هر دو معیار نشان می‌دهند که تقسیم پیشنهادی (D_1, D_2) بهبودی نسبت به گره ریشه ایجاد می‌کند — پس این تقسیم قابل قبول است. - معیار Gini به دلیل محذب بودن و حساسیت به تغییرات در احتمالات کوچک، در برخی موارد (مخصوصاً وقتی توزیع کلاس‌ها نامتقارن است) رفتار متفاوتی نسبت به misclassification error دارد. - در این مثال، چون توزیع اولیه کاملاً متقارن است و پس از تقسیم هم توزیع‌ها مشابه هستند، هر دو معیار روند یکسانی را نشان می‌دهند — اما مقدار عددی متفاوت است.

(ج) ماتریس هزینه confusion و توجیه

تعریف ماتریس هزینه

فرض کنید ماتریس هزینه $C \in \mathbb{R}^{4 \times 4}$ به صورت زیر تعریف شود (با فرض اینکه اشتباه طبقه‌بندی $c_i \rightarrow c_j$ هزینه‌ای دارد):

$$C = \begin{pmatrix} 0 & 1 & 2 & 3 \\ 1 & 0 & 1 & 2 \\ 2 & 1 & 0 & 1 \\ 3 & 2 & 1 & 0 \end{pmatrix}$$

که در آن: - قطر اصلی = * (طبقه‌بندی درست $\leftarrow$ هزینه صفر) - هزینه اشتباه با فاصله کلاس‌ها افزایش می‌یابد (مثلاً اشتباه $c_1 \rightarrow c_4$ گران‌تر از $c_1 \rightarrow c_2$ است).
این یک ماتریس متقارن و نامنفی است و نشان‌دهنده یک مسئله طبقه‌بندی با هزینه‌های نامتقارن است.

تعریف (۱)

$$\text{cost}(c' | c) = \begin{cases} \geq 0, & c' \neq c, c \in C, \\ 0, & \text{در غیر این صورت} \end{cases}$$

این تعریف دقیقاً مطلوب است چون: - هزینه اشتباه همیشه غیرمنفی است (اصول اولیه در نظریه تصمیم). - هزینه طبقه‌بندی درست صفر است — یعنی هیچ جریمه‌ای برای جواب صحیح وجود ندارد. - این اجازه می‌دهد که در الگوریتم‌هایی مثل *cost-sensitive learning*، معیارینه‌سازی را به صورت انتظار هزینه (expected cost) تعریف کنیم:

$$\text{Expected Cost}(S) = \sum_c p(c | S) \cdot \min_{c''} \sum_{c''} p(c'' | S) \cdot \text{cost}(c' | c'')$$

چرا شکاف سمت راست ترجیح داده می‌شود؟

با توجه به ماتریس هزینه بالا: - در شکاف سمت چپ (D_1): کلاس غالب c_2 است (۲ نمونه). اگر تمام نمونه‌ها را به c_2 طبقه‌بندی کنیم، هزینه اشتباه:

$$\text{cost} = 1 \cdot \text{cost}(c_2 | c_1) + 1 \cdot \text{cost}(c_2 | c_3) = 1 \cdot 1 + 1 \cdot 1 = 2$$

- در شکاف سمت راست (D_2): کلاس غالب c_4 است (۲ نمونه). هزینه اشتباه:

$$\text{cost} = 1 \cdot \text{cost}(c_4 | c_1) + 1 \cdot \text{cost}(c_4 | c_3) = 1 \cdot 3 + 1 \cdot 1 = 4$$

اما این فقط برای یک برگ است! در واقع، در الگوریتم‌های *cost-sensitive*، ما به جای انتخاب کلاس غالب، کلاسی را انتخاب می‌کنیم که حداقل انتظار هزینه را داشته باشد.

برای D_2 : - اگر به c_4 طبقه‌بندی کنیم: $E[\text{cost}] = \frac{1}{4}(3) + \frac{1}{4}(1) + \frac{2}{4}(0) = 1.0$ - اگر به c_3 طبقه‌بندی کنیم: $E[\text{cost}] = \frac{1}{4}(0) + \frac{1}{4}(1) + \frac{2}{4}(3) = 1.75$

پس c_4 بهترین انتخاب است و هزینه انتظاری آن 1.0 است.
اما در D_1 : - به c_2 : $E[\text{cost}] = \frac{1}{4}(1) + \frac{2}{4}(0) + \frac{1}{4}(1) = 0.5$ - به c_1 : $E[\text{cost}] = \frac{1}{4}(0) + \frac{2}{4}(1) + \frac{1}{4}(2) = 1.0$

$\Rightarrow$ هزینه انتظاری در D_1 کمتر است ($0.5 < 1.0$).

بنابراین، اگر سؤال بیان می‌کند که «در شکاف سمت راست ترجیح داده شده است»، احتمالاً به دلیل یک ماتریس هزینه غیرمتقارن دیگر است — مثلاً اگر اشتباه $c_1 \rightarrow c_2$ بسیار گران‌تر از $c_4 \rightarrow c_1$ باشد.
توصیه: در چنین مسائلی، ماتریس هزینه باید دقیقاً مشخص شود. اما اصل کلی این است که:

در حضور هزینه‌های نامتقارن، انتخاب کلاس بر اساس حداقل انتظار هزینه (*not max probability*) انجام می‌شود — و این می‌تواند منجر به انتخاب کلاسی شود که احتمال آن کمتر است، اما هزینه اشتباه آن کمتر است.

تمرین ۷

(الف) برچسب (label) تمام گره‌ها را با توجه به هزینه‌های طبقه‌بندی مشخص کنید و همچنین هزینه اشتباه طبقه‌بندی را از استفاده از تابع هزینه کنید:

$$\text{cost}(c' | c) = \begin{cases} 1, & c' \neq c, c \in C, \\ 0, & \text{غیر این صورت} \end{cases} \quad (۲)$$

(ب) یک تابع هزینه جدید طراحی کنید که هیچ‌کدام از قارچ‌های سمی (toxic/poisonous) را به عنوان خوراکی (edible) طبقه‌بندی نکند. در استفاده از این تابع هزینه، برچسب ریشه (root) چه خواهد بود؟ آیا جواب (node) تغییر می‌کند؟

داده‌ها:

Color	Size	Points	Eatability
red	small	yes	toxic
brown	small	no	edible
brown	large	yes	edible
green	small	no	edible
red	large	no	edible

و ساختار درخت پیشنهادی:

$$\{\text{size attribute:}\} \xrightarrow{\text{small}} ? \text{ label:} ; \xrightarrow{\text{large}} ? \text{ label:}$$

حل

پیش‌فرض‌ها

- مجموعه کلاس‌ها: $C = \{\text{edible}, \text{toxic}\}$ - تعداد کل نمونه‌ها: $n = 5$ - توزیع کلاس‌ها:

$$\#(\text{toxic}) = 1, \quad \#(\text{edible}) = 4$$

(الف) برچسب‌گذاری گره‌ها با تابع هزینه (۲)

تابع هزینه (۲) یک تابع مقارن و **binary**- است: - هر اشتباه (چه $\text{edible} \leftarrow \text{toxic}$ ، چه $\text{toxic} \leftarrow \text{edible}$) هزینه ۱ دارد. - طبقه‌بندی درست: هزینه ۰.

در این حالت، برچسب یک گره را به صورت **کلاس غالب** (majority class) انتخاب می‌کنیم — چون هزینه اشتباه برای هر کلاس یکسان است.

گره ریشه (همه ۵ نمونه)

- $\text{edible: } ۴$ ، $\text{toxic: } ۱$ ← کلاس غالب = **edible** - هزینه اشتباه اگر به edible برچسب بزنیم:

$$\text{Expected Cost} = \frac{1}{5} \cdot \text{cost}(\text{edible} | \text{toxic}) = \frac{1}{5} \cdot 1 = 0.2$$

- اگر به toxic برچسب بزنیم:

$$\frac{4}{5} \cdot 1 = 0.8 > 0.2$$

← پس برچسب ریشه: edible

زیرگروه $\text{small} = \text{size}$

نمونه‌هایی که $\text{small} = \text{size}$ هستند:

Eatability	Points	Size	Color
toxic	yes	small	red
edible	no	small	brown
edible	no	small	green

$\Rightarrow \#(\text{edible}) = 2, \#(\text{toxic}) = 1$

← کلاس غالب edible هزینه اشتباه: $1/3 \approx 0.333$ پس برچسب: edible

زیرگروه $\text{large} = \text{size}$

نمونه‌هایی که $\text{large} = \text{size}$ هستند:

Eatability	Points	Size	Color
edible	yes	large	brown
edible	no	large	red

$\Rightarrow \#(\text{edible}) = 2, \#(\text{toxic}) = 0$

← کلاس غالب edible هزینه اشتباه = ۰ پس برچسب: edible

نتیجه (الف):

همه گره‌ها برچسب edible می‌گیرند، زیرا در همه زیرمجموعه‌ها، edible اکثریت دارد و هزینه اشتباه برای هر دو نوع خطا یکسان است.

(ب) تابع هزینه جدید: جلوگیری از طبقه‌بندی toxic به edible

در این حالت، ما می‌خواهیم:

$$\text{cost}(\text{edible} | \text{toxic}) \gg \text{cost}(\text{toxic} | \text{edible})$$

$$\text{cost}(c' | c) = \begin{cases} 100, & c = \text{toxic}, c' = \text{edible} \quad (\text{اشتباه فاجعه‌بار}) \\ 1, & c = \text{edible}, c' = \text{toxic} \\ 0, & c' = c \end{cases}$$

(عدد ۱۰۰ دلخواه است — هر عدد بزرگی مثل ۱۰، ۵۰، ۱۰۰۰ قابل قبول است؛ مهم این است که بسیار بزرگتر از ۱ باشد.)

برچسب‌گذاری با تابع هزینه جدید

گره ریشه (۵ نمونه) - اگر برچسب edible : هزینه = $1 \cdot 100 = 100$ - اگر برچسب toxic : هزینه = $4 \cdot 1 = 4$
 ← چون $100 \gg 4$ ، برچسب بهینه اکنون: toxic

زیرگروه $\text{small} = \text{size}$ (۳ نمونه: ۲ edible ، ۱ toxic) - برچسب edible : هزینه = $1 \cdot 100 = 100$ - برچسب toxic : هزینه = $2 \cdot 1 = 2$ ← برچسب: toxic

زیرگره large (۲ نمونه: هر دو edible) - برچسب = edible: هزینه = * - برچسب = toxic: هزینه =
 $2 = 1 \cdot 2 \leftarrow$ برچسب: edible

نتیجه (ب): - با تابع هزینه جدید، برچسب ریشه از edible به toxic تغییر می‌کند. - این تغییر به دلیل این است که هزینه اشتباه $toxic \leftarrow edible$ بسیار بالا است، بنابراین الگوریتم ترجیح می‌دهد همه را toxic فرض کند تا risk این خطر فاجعه‌بار را بپذیرد. - در زیرگره‌هایی که toxic وجود ندارد (مثل size = large)، همچنان edible انتخاب می‌شود — چون هزینه اشتباه برای $edible \leftarrow toxic$ کوچک است.

آیا جواب (node) تغییر می‌کند؟

بله. در تابع هزینه اول (مقارن)، همه گره‌ها edible بودند. در تابع هزینه دوم (نا مقارن)، گره ریشه و زیرگره small به toxic تغییر کردند. نشان‌دهنده این است که **انتخاب برچسب به شدت به تابع هزینه وابسته است** — و در کاربردهای حیاتی (مثل تشخیص سمیت غذا)، باید تابع هزینه را با دقت طراحی کرد.

تمرین ۸

یک ماتریس 4×4 از نقاط با مختصات (x_1, x_2) که $x_1, x_2 \in \{0, 1, 2, 3, 4\}$ داده شده است. هر سلول دارای برچسب $+1$ یا -1 است:

$x_2 \backslash x_1$	۰	۱	۲	۳
۴	+1	-1	+1	-1
۳	-1	+1	-1	+1
۲	+1	-1	+1	-1
۱	-1	+1	-1	+1

(توجه: محور x_1 افقی و x_2 عمودی است؛ مبدأ در پایین چپ.)

(الف) حداقل چند معیار (condition) برای توصیف تمام نقاط با برچسب $+1$ لازم است؟ آیا می‌توان از یک شرط ساده مثل $x_1 < c$ یا $x_2 < c$ استفاده کرد؟ چرا یا چرا نه؟

(ب) حداقل چند معیار برای توصیف نقاط با برچسب $+1$ در بازه $0 < x_1 < 4$ لازم است؟ آیا می‌توان از یک شرط ساده مثل $x_1 + x_2 < c$ یا $x_1 \bmod 2 = x_2 \bmod 2$ استفاده کرد؟ و این شرط چه رابطه‌ای با الگوی مشاهده شده دارد؟

(ج) حداقل چند معیار برای توصیف نقاط با برچسب $+1$ در بازه $x_1 > 3$ لازم است؟ آیا می‌توان از یک شرط ساده مثل $x_2 > c$ استفاده کرد؟ اگر اجازه داشته باشیم از هر دو ویژگی (x_1, x_2) استفاده کنیم، چه شرطی می‌تواند تمام نقاط $+1$ را پوشش دهد؟

حل

تحلیل الگو

ابتدا مختصات تمام نقاط با برچسب $+1$ را استخراج می‌کنیم (با فرض که مختصات از $(0, 1)$ تا $(3, 4)$ هستند — چون در جدول، $x_1 = 0, 1, 2, 3$ و $x_2 = 1, 2, 3, 4$):

$$(+1): (0, 4), (2, 4), (1, 3), (3, 3), (0, 2), (2, 2), (1, 1), (3, 1)$$

مشاهده می‌شود که: - در هر سطر، برچسب‌ها به صورت ****مشبکه‌ای (checkerboard)**** تغییر می‌کنند. - به عبارت دیگر:

$$y = +1 \iff (x_1 + x_2) \equiv 0 \pmod{2}$$

یعنی مجموع مختصات زوج است. بررسی:

(x_1, x_2)	$x_1 + x_2$	زوج؟
(0, 4)	4	بله $\rightarrow +1$
(2, 4)	6	بله $\rightarrow +1$
(1, 3)	4	بله $\rightarrow +1$
(3, 3)	6	بله $\rightarrow +1$
(0, 2)	2	بله $\rightarrow +1$
(2, 2)	4	بله $\rightarrow +1$
(1, 1)	2	بله $\rightarrow +1$
(3, 1)	4	بله $\rightarrow +1$

در حالی که برای -1 :

... , فرد 7: $(3, 4)$, فرد 5: $(1, 4)$

پس الگوی دقیق:

$$y = +1 \iff (x_1 + x_2) \bmod 2 = 0$$

(الف) حداقل تعداد معیارها برای توصیف +1

اگر فقط از شرط‌های ساده نوع $x_1 < c$ یا $x_2 < c$ استفاده کنیم (یعنی hyperplanes محوری)، آیا می‌توانیم تمام نقاط +1 را پوشش دهیم؟

- شرط $x_1 < c$: یک نیم‌صفحه افقی — نمی‌تواند الگوی شبکه‌ای را توصیف کند. - شرط $x_2 < c$: یک نیم‌صفحه عمودی — همینطور. - حتی ترکیب چند شرط از این نوع (مثل $x_1 < 2 \wedge x_2 > 2$) فقط مستطیل‌های محوری ایجاد می‌کند، نه الگوی شطرنجی.

نتیجه: برای توصیف دقیق تمام نقاط +1 با شرط‌های محوری حداقل $\square$ (axis-aligned) **چهار شرط** لازم است — مثلاً:

$$\begin{cases} (x_1 = 0 \wedge x_2 \in \{2, 4\}) \\ (x_1 = 1 \wedge x_2 \in \{1, 3\}) \\ (x_1 = 2 \wedge x_2 \in \{2, 4\}) \\ (x_1 = 3 \wedge x_2 \in \{1, 3\}) \end{cases}$$

یا به صورت DNF:

$$(x_1 = 0 \wedge x_2 = 2) \vee (x_1 = 0 \wedge x_2 = 4) \vee \dots \vee (x_1 = 3 \wedge x_2 = 3)$$

که ۸ عبارت دارد — اما با گروه‌بندی، می‌توان به ۴ شرط کاهش داد.

اما اگر از شرط‌های غیرمحوری مانند $x_1 + x_2 \bmod 2 = 0$ استفاده کنیم، تنها **یک شرط** کافی است. پاسخ (الف): - با شرط‌های محوری (مثل $x_2 < c, x_1 < c$): حداقل **۴ شرط** لازم است (و نمی‌توان با یک شرط ساده این کار را کرد). - دلیل: الگوی شبکه‌ای غیرقابل برش با خطوط افقی/عمودی است.

(ب) در بازه $0 < x_1 < 4$ (یعنی $x_1 = 1, 2, 3$)

نقاط +1 در این بازه:

$$(1, 3), (3, 3), (2, 2), (1, 1), (3, 1)$$

الگو همچنان: $x_1 + x_2$ زوج.

آیا می‌توان از یک شرط ساده استفاده کرد؟ بله:

$$x_1 \bmod 2 = x_2 \bmod 2$$

یا معادل آن:

$$(x_1 + x_2) \bmod 2 = 0$$

این شرط تمام ۵ نقطه +1 را پوشش می‌دهد و هیچ نقطه -1 را شامل نمی‌شود (در این بازه).

پاسخ (ب): - حداقل **یک شرط** کافی است: $x_1 \bmod 2 = x_2 \bmod 2$. - این شرط دقیقاً الگوی شبکه‌ای را توصیف می‌کند — یعنی نقاط روی قطرهای موازی با خط ثابت $x_1 + x_2$.

(ج) در بازه $x_1 > 3$ (یعنی $x_1 = 4$)

اما در جدول اصلی، x_1 فقط تا ۳ است! پس احتمالاً منظور $x_1 = 3$ (چون در جدول، ستون آخر $x_1 = 3$ است) یا اشتباه تایپی است.

با فرض اینکه « $x_1 > 3$ » یعنی $x_1 = 4$ (که خارج از داده است)، ولی اگر منظور **ستون $x_1 = 3$ ** باشد (چون در جدول، ستون آخر $x_1 = 3$ است)، آنگاه:

نقاط در $x_1 = 3$:

$$(3, 4) : -1, \quad (3, 3) : +1, \quad (3, 2) : -1, \quad (3, 1) : +1$$

پس نقاط $(3, 3), (3, 1)$: $+1$ و $(3, 4), (3, 2)$: -1 استفاده کرد؟ بله: - اگر $c = 2$: $x_2 = 3, 4$: $x_2 > 2 \Rightarrow x_2 = 3, 4$ شامل
 $(3, 3)$ و $(3, 4)$ اما $(3, 4)$ برچسب -1 دارد خطا. - اگر $c = 1$: $x_2 = 2, 3, 4$: $x_2 > 1 \Rightarrow x_2 = 2, 3, 4$ شامل
 $(3, 2), (3, 3), (3, 4)$ دو تا -1 دارد. - اگر $c = 0$: همه را شامل می شود.
 پس با شرط های نوع $x_2 > c$ alone نمی توان فقط $(3, 3)$ و $(3, 1)$ را انتخاب کرد — چون این دو نقطه در
 $x_2 = 3$ و $x_2 = 1$ هستند (فاصله دارند).
 اما اگر از ترکیب دو ویژگی ** استفاده کنیم، می توانیم بنویسیم:

$$x_1 = 3 \wedge (x_2 = 1 \vee x_2 = 3)$$

یا بهتر:

$$x_1 = 3 \wedge (x_2 \bmod 2 = 1)$$

که فقط دو شرط اصلی دارد: $x_1 = 3$ و $x_2 \bmod 2 = 1$.
 پاسخ (ج): - با شرط های تک ویژگی (مثل $x_2 > c$): نمی توان ** با یک شرط تمام نقاط $+1$ را پوشش داد.
 - با استفاده از هر دو ویژگی: یک شرط مرکب کافی است:

$$x_1 = 3 \wedge (x_2 \bmod 2 = 1)$$

یا به صورت کلی تر:

$$(x_1 + x_2) \bmod 2 = 0 \quad \text{و} \quad x_1 = 3$$

جمع بندی نهایی

الگوی داده ها یک **مشبکه شطرنجی** است که فقط با شرط هایی بر اساس **پاریتی مجموع مختصات** قابل
 توصیف است. این نشان می دهد که: - الگوریتم هایی مانند **Perceptron** یا **Decision** (یا Stump) با
 شرط های محوری نمی توانند این الگو را یاد بگیرند. - اما الگوریتم هایی که از ویژگی های ترکیبی (مثل $x_1 + x_2$)
 یا ویژگی های غیرخطی پشتیبانی می کنند (مثل SVM با kernel یا درخت تصمیم با شرط های پیچیده)، می توانند آن
 را یاد بگیرند.

تمرین ۹

فرض کنید الگوریتم AdaBoost روی یک مجموعه آموزشی از اندازه m اجرا شود و فرض کنید هر دور (round) یک مدل ضعیف (t -امین مدل) با دقت بالاتر از $\frac{1}{2} - \gamma$ (برای $0 < \gamma$ ثابت) ارائه دهد، یعنی:

$$h_t \geq \frac{1}{2} - \gamma \iff \varepsilon_t \leq \frac{1}{2} + \gamma,$$

که ε_t خطای وزن دار مدل h_t است. نشان دهید که اگر تعداد دورها T بزرگتر از

$$T > \frac{\ln m}{2\gamma^2} \quad (۳)$$

باشد، آنگاه مدل نهایی AdaBoost بر روی مجموعه آموزشی **سازگار** (consistent) خواهد بود — یعنی خطای آموزشی صفر است. همچنین، توضیح دهید که چرا این شرط نشان دهنده این است که AdaBoost تحت فرض «وجود مدل ضعیف با دقت ثابت»، می تواند به یک مدل قوی و سازگار همگرا شود.

حل

مرور کوتاه بر AdaBoost

در AdaBoost: در هر دور t ، یک مدل ضعیف $h_t : \mathcal{X} \rightarrow \{-1, +1\}$ آموزش داده می شود. - خطای وزن دار آن:

$$\varepsilon_t = \sum_{i=1}^m w_i^{(t)} \cdot \mathbb{I}(h_t(x_i) \neq y_i)$$

- ضریب وزن مدل:

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \varepsilon_t}{\varepsilon_t} \right)$$

- مدل نهایی:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$$

خطای آموزشی نهایی:

$$\text{Error Train} = \frac{1}{m} \sum_{i=1}^m \mathbb{I}(H(x_i) \neq y_i)$$

کلید اثبات: حدای خطای آموزشی AdaBoost

یک نتیجه معروف در تئوری AdaBoost (Freund & Schapire, ۱۹۹۷) می گوید:

$$\text{Error Train} \leq \frac{1}{m} \sum_{i=1}^m \exp \left(-y_i \sum_{t=1}^T \alpha_t h_t(x_i) \right) = \frac{1}{m} \prod_{t=1}^T Z_t,$$

که در آن $Z_t = \sum_{i=1}^m w_i^{(t+1)} = \sum_i w_i^{(t)} \exp(-\alpha_t y_i h_t(x_i))$ و می توان نشان داد:

$$Z_t = 2\sqrt{\varepsilon_t(1 - \varepsilon_t)}.$$

بنابراین:

$$\text{Error Train} \leq \prod_{t=1}^T 2\sqrt{\varepsilon_t(1-\varepsilon_t)}.$$

حال، فرض می‌کنیم که برای همه t :

$$\varepsilon_t \leq \frac{1}{2} + \gamma \quad \text{و} \quad 1 - \varepsilon_t \geq \frac{1}{2} - \gamma.$$

پس:

$$\varepsilon_t(1-\varepsilon_t) \leq \left(\frac{1}{2} + \gamma\right) \left(\frac{1}{2} - \gamma\right) = \frac{1}{4} - \gamma^2.$$

در نتیجه:

$$Z_t = 2\sqrt{\varepsilon_t(1-\varepsilon_t)} \leq 2\sqrt{\frac{1}{4} - \gamma^2} = \sqrt{1 - 4\gamma^2}.$$

از آنجا که برای $1 \ll \gamma$ داریم $\sqrt{1 - 4\gamma^2} \leq e^{-2\gamma^2}$ (چون $1 - x \leq e^{-x}$ و $\sqrt{1-x} \leq e^{-x/2}$)، پس:

$$Z_t \leq e^{-2\gamma^2}.$$

بنابراین:

$$\text{Error Train} \leq \prod_{t=1}^T Z_t \leq \left(e^{-\gamma^2}\right)^T = e^{-2\gamma^2 T}.$$

برای اینکه خطای آموزشی **صفر** باشد، کافی است:

$$e^{-2\gamma^2 T} < \frac{1}{m} \quad \Longleftrightarrow \quad -2\gamma^2 T < -\ln m \quad \Longleftrightarrow \quad T > \frac{\ln m}{2\gamma^2}.$$

که دقیقاً نابرابری (۳) است.

تفسیر نتیجه

- این نشان می‌دهد که اگر در هر دور، مدل ضعیف حتی فقط به اندازه $\gamma > 0$ بهتر از حد تصادفی (0.5) عمل کند، آنگاه با افزایش تعداد دورها T ، خطای آموزشی به صفر میل می‌کند. - این خاصیت را **سازگاری** (consistency) می‌نامند: مدل نهایی روی داده‌های آموزشی بدون خطا پیش‌بینی می‌کند. - شرط $T > \frac{\ln m}{2\gamma^2}$ نشان می‌دهد که تعداد لازم دورها به طور لوگاریتمی با اندازه داده رشد می‌کند — یعنی حتی برای داده‌های بزرگ، تعداد دورهای مورد نیاز محدود است (اگر γ ثابت باشد).

نکته مهم: مفهوم «مدل ضعیف»

- در AdaBoost «مدل ضعیف» به معنای مدلی است که دقت آن **به طور یکنواخت** از 0.5 بیشتر باشد — نه اینکه فقط در برخی نمونه‌ها خوب باشد. - این فرض کلیدی است: اگر گاهی مدل ضعیف دقت < 0.5 داشته باشد، AdaBoost ممکن است واگرا شود. - در عمل، این فرض در بسیاری از مسائل واقعی برقرار است (مثلاً درخت‌های تصمیم کوچک با عمق ۱ — decision stumps — معمولاً دقتی حدود 0.55–0.6 دارند).

تمرین ۱۰

فرض کنید یک مجموعه آموزشی از اندازه m داده شده است و الگوریتم $bagging$ (Bootstrap Aggregating) روی آن اجرا شود. در هر تکرار، یک نمونه گیری با جایگذاری از مجموعه اصلی انجام می شود و یک مدل فردی (مثلاً درخت تصمیم) آموزش داده می شود. سپس پیش بینی نهایی با میانگین گیری (برای regression) یا رأی اکثریت (برای classification) انجام می شود.

۱. آیا با افزایش $m \rightarrow \infty$ ، خطای میانگین مدل های فردی (individual models) کاهش می یابد؟ چرا؟
۲. آیا خطای مدل نهایی Bagging (یعنی میانگین/رأی اکثریت) هم کاهش می یابد؟ اگر بله، چرا؟ اگر خیر، چه اتفاقی می افتد؟
۳. چرا Bagging می تواند از بیش برازش (overfitting) جلوگیری کند، حتی اگر مدل های فردی بیش برازش شده باشند؟

حل

مرور کوتاه بر Bagging

در Bagging - برای $b = 1, 2, \dots, B$ (تعداد مدل ها):

$$\mathcal{D}_b \sim \text{Bootstrap}(\mathcal{D}), \quad h_b = \text{Learn}(\mathcal{D}_b)$$

- مدل نهایی:

$$H(x) = \begin{cases} \frac{1}{B} \sum_{b=1}^B h_b(x) & (\text{Regression}) \\ \text{mode}\{h_1(x), \dots, h_B(x)\} & (\text{Classification}) \end{cases}$$

کلید اصلی: مدل های فردی ناهمبسته (یا کم همبسته) هستند، زیرا روی نمونه های متفاوتی آموزش داده می شوند.

۱. رفتار مدل های فردی وقتی $m \rightarrow \infty$

فرض کنید مدل فردی h_b یک الگوریتم یادگیری باشد که تحت شرایط مناسب (مثلاً یک درخت تصمیم با عمق محدود)، سازگار (consistent) است — یعنی:

$$\lim_{m \rightarrow \infty} \mathbb{E}_{\mathcal{D}} [\text{Error}(h_b)] = 0.$$

اما در Bagging، هر مدل h_b روی یک نمونه گیری با جایگذاری از $\mathcal{D}$ آموزش داده می شود. اندازه هر نمونه بوت استرپ $0.632m \approx$ است (چون انتظار تعداد نقاط منحصر به فرد در نمونه بوت استرپ برابر $m(1 - e^{-1}) \approx 0.632m$ است).

بنابراین: - وقتی $m \rightarrow \infty$ ، اندازه هر نمونه بوت استرپ هم $\rightarrow \infty$ - اگر الگوریتم فردی (مثلاً درخت با عمق ثابت) سازگار باشد، آنگاه خطای هر h_b به صفر میل می کند:

$$\forall b : \lim_{m \rightarrow \infty} \mathbb{E}[\text{Error}(h_b)] = 0.$$

پاسخ قسمت ۱: بله، خطای مدل های فردی با افزایش m کاهش می یابد — به شرطی که الگوریتم فردی سازگار باشد (مثل درخت های با عمق محدود، یا رگرسیون خطی).

۲. رفتار مدل نهایی Bagging وقتی $m \rightarrow \infty$

خطای مدل نهایی Bagging برای مسئله طبقه‌بندی به صورت زیر تجزیه می‌شود (بر اساس تحلیل واریانس-سوگیری):

$$\mathbb{E}[\text{Error}(H)] = \underbrace{\text{Bias}^2}_{\text{ثابت}} + \underbrace{\text{Variance}}_{\downarrow \text{ با } B \text{ و } m}$$

اما در حالت $m \rightarrow \infty$: هر مدل فردی h_b به یک مدل حدی h^* همگرا می‌شود (که بر اساس تمام داده‌ها آموزش داده شده). — اما چون نمونه‌های بوت‌استرپ متفاوت هستند، مدل‌های h_b همچنان کمی متفاوت هستند — اما اختلاف آن‌ها به صفر میل می‌کند. به طور دقیق‌تر، برای $m \rightarrow \infty$:

$$h_b(x) \xrightarrow{p} h^*(x) \quad \text{برای } b,$$

پس:

$$H(x) = \frac{1}{B} \sum_{b=1}^B h_b(x) \xrightarrow{p} h^*(x).$$

و اگر h^* خود سازگار باشد (یعنی $\text{Error}(h^*) \rightarrow 0$ وقتی $m \rightarrow \infty$)، آنگاه:

$$\lim_{m \rightarrow \infty} \mathbb{E}[\text{Error}(H)] = 0.$$

پاسخ قسمت ۲: بله، خطای مدل نهایی Bagging هم به صفر میل می‌کند — زیرا هم مدل‌های فردی همگرا می‌شوند و میانگین آن‌ها هم به همان حد همگرا می‌شود.

۳. چرا Bagging از بیش‌برازش جلوگیری می‌کند؟

این یکی از قدرت‌های اصلی Bagging است:

— فرض کنید هر مدل فردی h_b یک درخت تصمیم کامل (بدون pruning) باشد — یعنی بیش‌برازش شده است (high variance, low bias). — هر h_b روی یک نمونه بوت‌استرپ آموزش داده شده و به دلیل نویز داده، پیش‌بینی‌های آن نوسان زیادی دارد. — اما چون نمونه‌های بوت‌استرپ مستقل هستند، نوسانات (variance) مدل‌های مختلف **ناهمبسته** هستند. — در میانگین‌گیری:

$$\text{Var} \left(\frac{1}{B} \sum_{b=1}^B h_b(x) \right) = \frac{1}{B^2} \sum_{b=1}^B \text{Var}(h_b(x)) + \frac{2}{B^2} \sum_{i < j} \text{Cov}(h_i, h_j)$$

اگر ک^{۴۴۴۴}یانس‌ها کوچک باشند (مثل Bagging)، آنگاه واریانس کلی تقریباً $\frac{1}{B}$ برابر واریانس یک مدل فردی می‌شود.

پس: — Bagging واریانس را کاهش می‌دهد — حتی اگر هر مدل فردی بیش‌برازش شده باشد. — این را می‌توان با مثال عددی دید: فرض کنید ۳ مدل فردی پیش‌بینی‌های $[-1, +1, +1]$ برای یک نمونه داشته باشند (در حالی که برچسب واقعی $+1$ است). $\leftarrow$ رأی اکثریت: $+1$ (درست)، در حالی که یکی از مدل‌ها اشتباه است.

پاسخ قسمت ۳: Bagging با میانگین‌گیری از مدل‌های ناهمبسته، واریانس کلی را کاهش می‌دهد و از بیش‌برازش جلوگیری می‌کند — حتی اگر هر مدل فردی به تنهایی بیش‌برازش شده باشد.

جمع‌بندی نهایی

- با افزایش $m \rightarrow \infty$ ، هم مدل‌های فردی و هم مدل نهایی Bagging به یک مدل سازگار همگرا می‌شوند.

- Bagging یک روش کاهش واریانس است، نه کاهش سوگیری — بنابراین برای مدل‌هایی که واریانس بالایی دارند (مثل درخت‌های عمیق)، بسیار مؤثر است.
- این خاصیت، تفاوت اصلی Bagging با روش‌هایی مثل Boosting است: Boosting سعی می‌کند سوگیری را کاهش دهد (با تمرکز روی نمونه‌های سخت)، در حالی که Bagging واریانس را کاهش می‌دهد.

تمرین ۱۱

داده‌های دو کلاسی (positive class و negative class) در فضای دو بعدی داده شده‌اند (نمودار چپ و راست). هدف: پروجکت کردن داده‌ها به یک بعد برای طبقه‌بندی.

(الف) اگر PCA استفاده کنیم، جهت پروجکت (projection direction) چیست؟ آیا این جهت برای جدا سازی کلاس‌ها مناسب است؟ چرا؟

(ب) اگر از Fisher's Linear Discriminant (FLD) استفاده کنیم، جهت پروجکت چیست؟ آیا این جهت برای جدا سازی کلاس‌ها مناسب است؟ چرا؟

(ج) چرا PCA در برخی موارد (مثل این مثال) منجر به اتلاف اطلاعات برای طبقه‌بندی می‌شود، در حالی که FLD هدفمند است؟

(د) اگر داده‌ها را با یک تبدیل خطی (مثل دوران) تغییر دهیم، آیا جهت PCA تغییر می‌کند؟ جهت FLD چه؟

حل

مرور کوتاه بر PCA و FLD

• **PCA** (Principal Component Analysis): جهت پروجکت را به گونه‌ای انتخاب می‌کند که واریانس داده‌های پروجکت شده حداکثر شود. ریاضیاً: بردار ویژه مربوط به بزرگترین مقدار ویژه ماتریس کوواریانس $S = \frac{1}{m} X^T X$ است.

• **Fisher's LDA** (Linear Discriminant Analysis): جهت پروجکت را به گونه‌ای انتخاب می‌کند که نسبت بین کلاسی به درون کلاسی (between-class to within-class scatter) حداکثر شود:

$$w^* = \arg \max_w \frac{w^T S_B w}{w^T S_W w},$$

که:

$$S_B = (\mu_+ - \mu_-)(m\mu_+ - m\mu_-)^T, \quad S_W = \sum_{c \in \{+, -\}} \sum_{x \in C_c} (x - \mu_c)(x - \mu_c)^T.$$

(الف) PCA و جهت پروجکت

با توجه به نمودار چپ (که داده‌های positive و negative را نشانی دهد): - داده‌ها در جهتی افقی-مایل پخش شده‌اند (مثلاً از چپ‌پایین به راست‌بالا). - واریانس کلی در جهت اصلی (بردار اول PCA) حداکثر است — احتمالاً جهتی نزدیک به خط $y = x$ یا $y = 0.5x$.

اما این جهت برای جدا سازی کلاس‌ها مناسب نیست، زیرا: - دو کلاس در این جهت همپوشانی زیادی دارند. - PCA به برچسب‌ها توجه نمی‌کند — فقط واریانس را ماکزیمم می‌کند.

پاسخ (الف): جهت PCA = جهت بیشترین واریانس (معمولاً افقی-مایل در این نمودار). خیر، این جهت برای جدا سازی کلاس‌ها مناسب نیست، زیرا PCA از اطلاعات برچسب استفاده نمی‌کند.

(ب) FLD و جهت پروجکت

در نمودار راست، داده‌ها به گونه‌ای هستند که: - میانگین کلاس مثبت (μ_+) در سمت راست بالا و میانگین کلاس منفی (μ_-) در سمت چپ پایین قرار دارد. - فاصله بین میانگین‌ها در جهتی عمود بر خط اتصال میانگین‌ها کم است، اما در جهت اتصال میانگین‌ها (یعنی $\mu_+ - \mu_-$) بیشتر است.

FLD دقیقاً جهت $\mu_+ - \mu_-$ را انتخاب می‌کند — چون:

$$S_B = (\mu_+ - \mu_-)(\mu_+ - \mu_-)^T \Rightarrow \text{است } \mu_+ - \mu_- \text{ همان } S_B \text{ بردار ویژه اصلی}$$

پس جهت FLD = جهت اتصال میانگین دو کلاس — که در این مثال، به صورت قطری از چپ‌پایین به راست بالا است.

و این جهت برای جدا سازی کلاس‌ها ایده‌آل است، زیرا: - پروجکت روی این جهت، دو کلاس را به طور ماکزیمم از هم جدا می‌کند. - واریانس درون‌کلاسی کم و فاصله بین‌کلاسی بیشتر می‌شود.

پاسخ (ب): جهت FLD = جهت $\mu_+ - \mu_-$ (اتصال میانگین دو کلاس). بله، این جهت برای جدا سازی کلاس‌ها بسیار مناسب است.

(ج) چرا PCA اطلاعات طبقه‌بندی را اتلاف می‌کند؟

PCA - یک روش **بدون نظارت** (unsupervised) است: تنها به ساختار داده (واریانس) توجه می‌کند، نه به برچسب‌ها. - در این مثال، واریانس اصلی در جهتی است که دو کلاس را همپوشان می‌کند — یعنی اطلاعاتی که برای طبقه‌بندی مفید است (تفاوت بین کلاس‌ها)، در جهت‌های ثانویه قرار دارد که PCA نادیده می‌گیرد. - در مقابل، FLD یک روش **با نظارت** (supervised) است و مستقیماً به دنبال جهتی است که کلاس‌ها را از هم جدا کند.

مثال عددی ساده: فرض کنید:

$$\mu_- = \begin{bmatrix} -1 \\ 0 \end{bmatrix}, \quad \mu_+ = \begin{bmatrix} +1 \\ 0 \end{bmatrix}, \quad \text{پخش شده‌اند } y \text{ اما داده‌ها در جهت}$$

← PCA جهت y را انتخاب می‌کند (واریانس بیشتر)، اما FLD جهت x را انتخاب می‌کند (فاصله بین کلاس‌ها).

(د) تأثیر تبدیل خطی (مثل دوران)

• PCA: اگر داده‌ها را دوران دهیم، ماتریس کوواریانس نیز دوران می‌یابد، و بردارهای ویژه آن هم دوران می‌کنند. ← بله، جهت PCA تغییر می‌کند (همراه با داده).

• FLD: اگر داده‌ها را با ماتریس دوران R تبدیل

$$x \mapsto Rx,$$

آنگاه:

$$\mu_c \mapsto R\mu_c, \quad S_W \mapsto RS_W R^T, \quad S_B \mapsto RS_B R^T.$$

و جهت بهینه:

$$w^* = \arg \max_w \frac{w^T S_B w}{w^T S_W w} \Rightarrow R w^* \text{ تبدیل شده}$$

← پس جهت FLD هم تغییر می‌کند، اما نسبت به داده‌ها همراه است — یعنی همیشه جهت اتصال میانگین‌ها را دنبال می‌کند.

نکته مهم: هر دو روش تحت تبدیل‌های خطی «کافاکانت» هستند — یعنی جهت‌ها با داده تغییر می‌کنند، اما رابطه آن‌ها با ساختار داده حفظ می‌شود.

جمع‌بندی نهایی

Fisher's LDA	PCA	ویژگی
با نظارت	بدون نظارت	نوع الگوریتم
ماکزیمم کردن نسبت بین/درون‌کلاسی	ماکزیمم کردن واریانس	هدف
بله	خیر (در مواردی مثل این مثال)	مناسب برای طبقه‌بندی؟
جهت تغییر می‌کند (همراه با داده)	جهت تغییر می‌کند (همراه با داده)	تأثیر تبدیل خطی

تمرین ۱۲

سه نقطه در فضای دو بعدی داده شده‌اند:

$$(-1, -1), \quad (0, 0), \quad (1, 1).$$

(الف) مؤلفه اصلی اول (first principal component) چیست؟ (بردار دقیق را بنویسید.)

(ب) اگر نقطه‌های داده‌شده را به یک زیرفضای 1-بعدی (یعنی خطی) پروجکت کنیم، آیا می‌توانیم داده‌ها را بازسازی کنیم؟ چرا؟

(ج) برای هر نقطه، خطای بازسازی (reconstruction error) را محاسبه کنید. آیا این خطا صفر است؟ چرا؟

حل

مرور کوتاه بر PCA

برای یک مجموعه داده $\{x_1, \dots, x_m\} \subset \mathbb{R}^d$

۱. مرکز داده‌ها را محاسبه کنید: $\mu = \frac{1}{m} \sum_{i=1}^m x_i$

۲. ماتریس کوواریانس: $S = \frac{1}{m} \sum_{i=1}^m (x_i - \mu)(x_i - \mu)^T$

۳. بردار اولین مؤلفه اصلی = بردار ویژه مربوط به بزرگترین مقدار ویژه S .

(الف) یافتن اولین مؤلفه اصلی

داده‌ها:

$$x_1 = \begin{bmatrix} -1 \\ -1 \end{bmatrix}, \quad x_2 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad x_3 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

مرکز داده‌ها

$$\mu = \frac{1}{3} \left(\begin{bmatrix} -1 \\ -1 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right) = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

← داده‌ها از قبل مرکز شده‌اند.

ماتریس کوواریانس

$$S = \frac{1}{3} \sum_{i=1}^3 x_i x_i^T = \frac{1}{3} \left(\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \right) = \frac{1}{3} \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix} = \begin{bmatrix} \frac{2}{3} & \frac{2}{3} \\ \frac{2}{3} & \frac{2}{3} \end{bmatrix}$$

مقادیر ویژه و بردارهای ویژه

معادله ویژه:

$$\det(S - \lambda I) = \begin{vmatrix} \frac{2}{3} - \lambda & \frac{2}{3} \\ \frac{2}{3} & \frac{2}{3} - \lambda \end{vmatrix} = \left(\frac{2}{3} - \lambda\right)^2 - \left(\frac{2}{3}\right)^2 = 0$$

$$\Rightarrow \left(\frac{2}{3} - \lambda - \frac{2}{3}\right) \left(\frac{2}{3} - \lambda + \frac{2}{3}\right) = (-) \left(\frac{4}{3} - \lambda\right) = 0$$

پس مقادیر ویژه:

$$\lambda_1 = \frac{4}{3}, \quad \lambda_2 = 0$$

بردار ویژه مربوط به $\lambda_1 = \frac{4}{3}$:

$$(S - \frac{4}{3}I)v = 0 \Rightarrow \begin{bmatrix} -\frac{2}{3} & \frac{2}{3} \\ \frac{2}{3} & -\frac{2}{3} \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = 0 \Rightarrow -v_1 + v_2 = 0 \Rightarrow v_1 = v_2$$

پس بردار ویژه (نرمال شده):

$$v_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

پاسخ (الف): اولین مؤلفه اصلی (به صورت بردار یک) است:

$$w = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

(ب) آیا بازسازی امکان پذیر است؟

پروجکت هر نقطه x_i روی زیرفضای 1- بعدی تولیدشده توسط w برابر است با:

$$\hat{x}_i = (w^T x_i)w$$

محاسبه برای هر نقطه:

$$\hat{x}_1 = (-\sqrt{2}) \cdot \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ -1 \end{bmatrix} = w^T x_1 = \frac{1}{\sqrt{2}}(-1 - 1) = -\frac{2}{\sqrt{2}} = -\sqrt{2} : x_1 = [-1, -1]^T \bullet$$

x_1

$$\hat{x}_2 = 0 \cdot w = [0, 0]^T = x_2 : x_2 = [0, 0]^T \bullet$$

$$\hat{x}_3 = \sqrt{2} \cdot \frac{1}{\sqrt{2}} [1, 1]^T = [1, 1]^T = x_3 : w^T x_3 = \frac{2}{\sqrt{2}} = \sqrt{2} : x_3 = [1, 1]^T \bullet$$

همه نقاط دقیقاً روی خط $y = x$ قرار دارند — یعنی داده‌ها از ابتدا در یک زیرفضای 1- بعدی هستند.
پاسخ (ب): بله، بازسازی کامل امکان پذیر است، زیرا تمام نقاط روی یک خط (زیرفضای 1- بعدی) قرار دارند و اولین مؤلفه اصلی همان جهت این خط است.

(ج) خطای بازسازی (Reconstruction Error)

خطای بازسازی برای هر نقطه:

$$\text{Reconstruction Error}_i = \|x_i - \hat{x}_i\|_2$$

از محاسبات بالا:

$$\|x_1 - \hat{x}_1\| = 0, \quad \|x_2 - \hat{x}_2\| = 0, \quad \|x_3 - \hat{x}_3\| = 0$$

پس خطای کلی (میانگین یا مجموع) صفر است.
پاسخ (ج): خطای بازسازی برای همه نقاط صفر است، زیرا داده‌ها از ابتدا در زیرفضای 1-بعدی قرار دارند و PCA دقیقاً آن زیرفضا را یافته است.

نکته مفهومی

- اگر داده‌ها در یک زیرفضای k -بعدی قرار داشته باشند، PCA با انتخاب k مؤلفه اصلی، بازسازی کامل (بدون خطا) انجام می‌دهد. - در این مثال، رتبه ماتریس داده‌ها برابر ۱ است (چون تمام بردارها ضربی از $[1, 1]^T$ هستند)، پس تنها یک مؤلفه اصلی لازم است.

تمرین ۱۳

چه رابطه‌ای بین روش‌های زیر وجود دارد؟ SVD، PCA، NMF، PPCA و Tucker. همچنین، بررسی کنید که هر کدام از این روش‌ها در چه شرایطی مناسب‌ترند و چه تفاوت‌هایی با هم دارند. آیا این روش‌ها قابلیت پیش‌بینی (prediction) دارند؟

حل

مرور کوتاه بر هر روش

۱. **SVD** (Singular Value Decomposition): برای هر ماتریس $X \in \mathbb{R}^{m \times n}$ ، تجزیه می‌شود:

$$X = U \Sigma V^T,$$

که U, V متعامد و Σ قطری با مقادیر منفرد $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$ است. نکته: SVD یک روش جبری خالص است — بدون فرض احتمالی.

۲. **PCA** (Principal Component Analysis): اگر X مرکز شده باشد، PCA معادل است با نگه داشتن k ستون اول U و k مقدار منفرد اول Σ_k در SVD:

$$X \approx U_k \Sigma_k V_k^T \Rightarrow \text{Projection: } Z = U_k^T X.$$

پس: SVD روی داده‌های مرکز شده PCA =

۳. **PPCA** (Probabilistic PCA): یک مدل احتمالی است که فرض می‌کند:

$$x = Wz + \mu + \epsilon, \quad z \sim \mathcal{N}(0, I), \quad \epsilon \sim \mathcal{N}(0, \sigma^2 I).$$

با بیشینه‌سازی likelihood می‌توان نشان داد که بردارهای W همان اولین k مؤلفه اصلی (از PCA) هستند — اما با یک پارامتر اضافه (σ^2) برای مدل‌سازی نویز. پس: PPCA $\rightarrow$ PCA احتمالی.

۴. **NMF** (Non-negative Matrix Factorization): فرض می‌کند $X \geq 0$ و تجزیه می‌شود:

$$X \approx WH, \quad W \geq 0, \quad H \geq 0.$$

برخلاف PCA/SVD/NMF محدودیت غیرمنفی دارد — که برای داده‌هایی مثل تصاویر، متن یا سیگنال‌های طیفی مناسب است (چون این داده‌ها طبیعتاً غیرمنفی هستند). تفاوت کلیدی: NMF از مفهوم «اجزای پایه» (parts-based representation) پشتیبانی می‌کند — در حالی که PCA از «الگوهای جمعی» (holistic patterns) استفاده می‌کند.

۵. **Tucker Decomposition**: برای تنسورهای مرتبه بالا (مثلاً $X \in \mathbb{R}^{I \times J \times K}$) تعریف می‌شود:

$$\mathcal{X} \approx \mathcal{G} \times_1 A \times_2 B \times_3 C,$$

که $\mathcal{G}$ تنسور کوچک (core tensor) و A, B, C ماتریس‌های تبدیل هستند. در حالت ماتریسی ($K = 1$)، Tucker به SVD تقلیل می‌یابد. پس: Tucker = تعمیم SVD به تنسورها.

رابطه‌های بین روش‌ها

رابطه با سایر روش‌ها	پایه ریاضی	روش
پایه، PCA، Tucker PPCA	جبر خطی	SVD
$\text{SVD} \equiv$ روی داده مرکز شده	تحلیل مؤلفه‌ها	PCA
تعمیم PCA: Bayesian PCA؛ نتیجه‌اش همان PCA است (اما با تفسیر احتمالی)	مدل احتمالی	PPCA
مستقل از SVD/PCA؛ فقط برای داده‌های غیرمنفی	بهینه‌سازی محدود	NMF
تعمیم SVD به ابعاد $2 <$	تنسورها	Tucker

کاربردها و شرایط مناسب

- **SVD/PCA**: وقتی داده‌ها مرکز شده‌اند و هدف کاهش ابعاد برای تجسم یا حذف نویز است (مثلاً در LSA، face recognition).
- **PPCA**: وقتی نیاز به مدل‌سازی نویز و محاسبه احتمال داریم (مثلاً در الگوریتم‌های Bayesian یا برای ارزیابی شانس داده‌ها).
- **NMF**: وقتی داده‌ها غیرمنفی هستند و می‌خواهیم «اجزای معنادار» (مثل topics در متن یا parts در تصویر) استخراج کنیم. مثال: LDA در متن، یا تحلیل میکروآرایه‌های ژنتیکی.
- **Tucker**: وقتی داده‌ها ساختار تنسوری دارند (مثلاً ویدیو = زمان $\times$ ارتفاع $\times$ عرض، یا ریزش سه‌بعدی در MRI). معمولاً در deep learning برای فشرده‌سازی لایه‌های شبکه استفاده می‌شود.

آیا این روش‌ها قابلیت پیش‌بینی دارند؟

- **Tucker / SVD / PCA**: خیر — این‌ها روش‌های توصیفی (descriptive) هستند. پس از کاهش ابعاد، می‌توان از مدل‌های دیگر (مثل SVM, kNN) برای پیش‌بینی استفاده کرد، اما خود PCA پیش‌بینی نمی‌کند.
- **PPCA**: بله — چون یک مدل احتمالی است، می‌توان از آن برای پیش‌بینی مقدارهای گمشده (missing data imputation) یا محاسبه احتمال $p(x)$ استفاده کرد.
- **NMF**: معمولاً خیر — اما در برخی پیاده‌سازی‌ها (مثل NMF با regularizer)، می‌توان از آن برای پیش‌بینی رتبه‌بندی (rating prediction در recommender systems) استفاده کرد — ولی این یک افزونه است، نه خاصیت ذاتی NMF.

جمع‌بندی نهایی

- PCA و SVD در واقع دو وجه یک سکه هستند — PCA یک کاربرد SVD است.
- PPCA نسخه احتمالی و قابل تفسیرتر PCA است.
- NMF برای داده‌های غیرمنفی و با تفسیر «اجزای پایه» منحصر به فرد است.
- Tucker تعمیم طبیعی SVD به تنسورهاست و در داده‌های چند بعدی ضروری است.
- تنها PPCA (و در برخی موارد NMF) قابلیت پیش‌بینی مستقیم دارد.